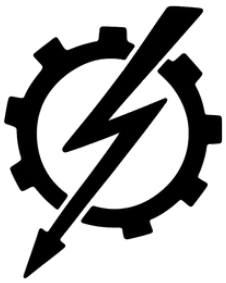
TWORZENIE APLIKACJI WEBOWYCH



```
|||> cat ${CURRENT_LAB}
```

```
<!DOCTYPE html>
<html>
<head>
```

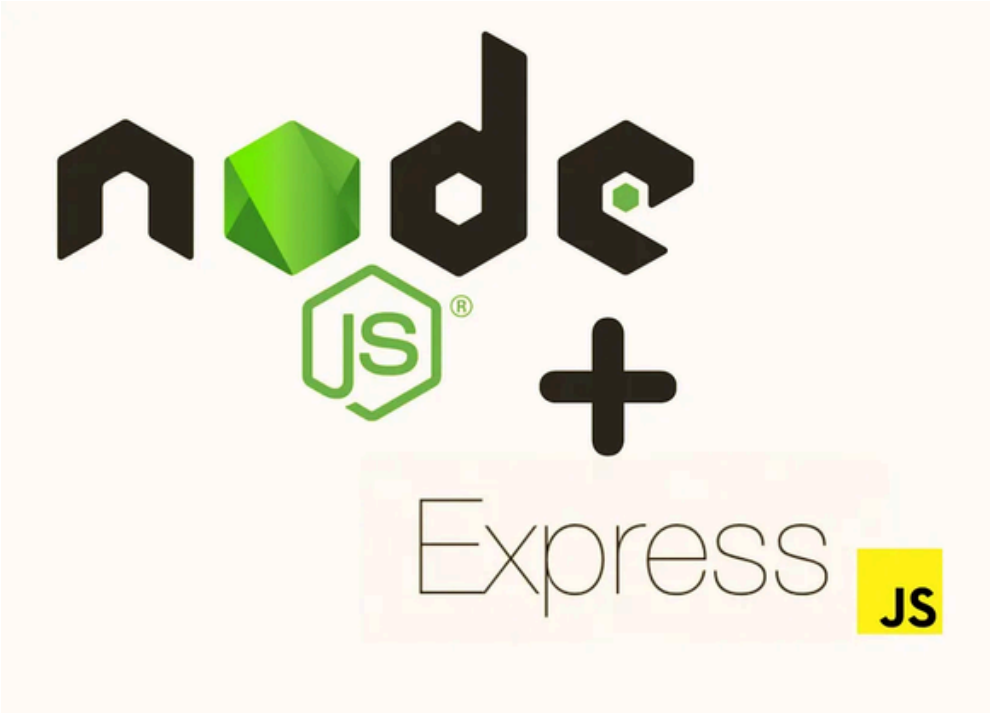
<title> WEB APPS </title>

```
</head>
<body>
```

<h1> LAB 3 </h1>

Express - routing & middleware

```
</body>
</html>
```




```
|||> echo “by ${AUTHOR}”
```

by Mateusz Węglewski

Task 3.0.

Przygotuj środowisko.

1. **Wejdź na *https://shell.cloud.google.com* i zaloguj się kontem Google.**
2. **Skonfiguruj swoją tożsamość w gicie: (może wystąpić konieczność powtórzenia tego)**

```
git config --global user.name "Imię Nazwisko"
git config --global user.email "twoj@email.com"
```

Task 3.1.

Sklonuj repozytorium i wejdź do środka.

```
cd ~
git clone https://github.com/mateuszweglewski/webapps_zsme_5.git
cd webapps_zsme_5/lab-express
```

Task 3.2.

Sprawdź konfigurację.

1. **Sprawdź wersję Node.js i npm:**

```
node -v
npm -v
```

Jeśli wersja Node jest niższa niż 20, wykonaj:

```
nvm install --lts
```

2. **Jaką wersję Node.js masz zainstalowaną? _____**

Task 3.3.

Wykonaj odpowiednie kroki i odpowiedz na pytania.

1. **Utwórz projekt Node i zainstaluj Express:**

```
npm init -y
ls
npm install express
ls
```

2. **Jaki plik utworzyło *npm init -y*? _____**
3. **Co nowego pojawiło się w folderze po *npm install express*?**

_____

4. **Dodaj plik *.gitignore* i skrypt uruchamiający:**

```
echo "node_modules/" > .gitignore
npm pkg set scripts.dev="node --watch app.js"
cat package.json
```

5. **Dlaczego *node_modules* wpisujemy do .gitignore, a *package-lock.json* wysyłamy do repozytorium?**

_____

Task 3.4.

Uruchom pierwszy serwer.

1. **Kliknij *Otwórz edytor*. Uzupełnij luki 1–6 i 14 w *app.js* oraz 15–16 w *middleware/logger.js*.**
2. **Uruchom serwer komendą *npm run dev*.**
3. **Otwórz drugą kartę terminala (+) i wykonaj:**

```
cd ~/lab-express
curl localhost:8080/api/czas
```

4. **Co pojawiło się w pierwszej karcie, w logu serwera? _____**

Task 3.5.

Routing.

1. **Uzupełnij luki 7–10 w *app.js* i zapisz plik.**
2. **Wykonaj:**

```
curl "localhost:8080/api/przedmioty?min=4"
curl -i localhost:8080/api/przedmioty/99
```

3. **Jaki kod statusu zwrócił drugi *curl*? _____**
4. **Czym różni się *req.params* od *req.query*?**

_____

Task 3.6.

POST i middleware z kluczem.

1. **Uzupełnij luki 12–13 w *app.js* i 17–18 w *middleware/wymagajKlucza.js*.**

W miejscu /* LUKA 11 */ **wpisz** wymagajKlucza,**. (tak, z przecinkiem.)**

2. **Wyślij przedmiot bez klucza, a potem z kluczem:**

```
curl -i -X POST localhost:8080/api/przedmioty -H "Content-Type: application/json" -d '{"nazwa":"Fizyka","godziny":2}'

curl -i -X POST localhost:8080/api/przedmioty -H "Content-Type: application/json" -H "x-api-key: lab123" -d '{"nazwa":"Fizyka","godziny":2}'
```

3. **Kod bez klucza: _____ Kod z kluczem: _____**

Task 3.7.

Sprawdź i oddaj.

1. **Przy działającym serwerze wykonaj w drugiej karcie:**

```
node sprawdz.js
```

2. **Wynik: _____ / 10. Pokaż go prowadzącemu.**

THE ENCORE

Dodaj DELETE /api/przedmioty/:id, chroniony kluczem. Po usunięciu ma zwracać *204*, a gdy przedmiotu nie ma, *404*.

Pokaż wyniki prowadzącemu.

Check your knowledge

- Node.js to:

  a) framework JavaScript b) środowisko uruchomieniowe JavaScript c) przeglądarka d) baza danych
- Które polecenie instaluje paczkę i dopisuje ją do dependencies?

  a) npm init -y b) npm run dev c) npm install express d) node express
- Dla adresu /api/przedmioty/5?min=3 podaj wartość:

  req.params.id = _____ req.query.min = _____
- Jaki kod statusu zwróci serwer, gdy:

  utworzono nowy zasób: _____ nie ma takiego zasobu: _____ brakuje klucza API: _____
- Middleware to funkcja z trzema argumentami: (_____, _____, _____).
- Co się stanie, jeśli middleware nie wywoła next() i nie wyśle odpowiedzi?
- Dlaczego middleware obsługujący 404 umieszczamy na samym końcu?

:wq